

Overhead de Operações Criptográficas em Bancos de Dados Distribuídos

Beatriz Viana Costa¹, Lys Katherine Park Kang¹,
Raphaella Brandão Jacques¹, Stephanie Maria Braga¹

¹Instituto de Matemática e Estatística (IME)
Universidade de São Paulo (USP) – São Paulo, SP – Brasil

Abstract. *As distributed systems increasingly store sensitive data, cryptographic techniques become essential to guarantee confidentiality and integrity. However, these techniques introduce performance costs that are amplified in distributed settings, where inter-node communication is already a natural bottleneck. This work investigates the impact of cryptographic operations on the performance of distributed databases, combining a literature review with practical experiments. We configure a distributed database cluster (Replica Set) and measure latency and throughput with and without encryption in transit (TLS), in order to quantify the introduced overhead and discuss the trade-offs between security and performance. Over a real, indexed, heterogeneous query workload on MongoDB, enabling cluster-wide TLS introduces a median latency overhead of about 16.7% (16.5% on average) and a throughput reduction of about 14.2% on average. This cost is dominated by the per-connection handshake and cluster topology discovery: it is nearly constant in absolute terms (≈ 110.6 ms per query), so the relative overhead shrinks as the intrinsic cost of the query grows.*

Resumo. *À medida que sistemas distribuídos passam a armazenar volumes crescentes de dados sensíveis, técnicas criptográficas tornam-se indispensáveis para garantir confidencialidade e integridade. Contudo, essas técnicas introduzem custos de desempenho que se tornam ainda mais relevantes em ambientes distribuídos, nos quais a comunicação entre nós já representa um gargalo natural. Este trabalho investiga o impacto de operações criptográficas sobre o desempenho de bancos de dados distribuídos, combinando uma revisão da literatura com experimentos práticos. Configuramos um cluster distribuído (Replica Set) e medimos latência e vazão com e sem criptografia em trânsito (TLS), a fim de quantificar o overhead introduzido e discutir os trade-offs entre segurança e desempenho. Sobre uma carga de trabalho real, indexada e heterogênea no MongoDB, habilitar o TLS no cluster introduz um overhead mediano de latência de cerca de 16,7% (16,5% em média) e uma queda de vazão média de cerca de 14,2%. Esse custo é dominado pelo handshake por conexão e pela descoberta de topologia do cluster: ele é quase constante em termos absolutos ($\approx 110,6$ ms por consulta), de modo que o overhead relativo diminui conforme cresce o custo intrínseco da consulta.*

1. Introdução

A crescente adoção de bancos de dados distribuídos para o armazenamento de grandes volumes de dados sensíveis torna a proteção criptográfica um requisito central de

projeto [Fuller et al. 2017]. Entretanto, garantir confidencialidade e integridade tem um custo: operações criptográficas consomem CPU e introduzem latência adicional, efeito agravado em sistemas distribuídos, nos quais a comunicação entre nós já constitui um gargalo natural [Gilbert and Lynch 2002].

Este trabalho tem como objetivo quantificar, de forma reproduzível, o overhead introduzido pela criptografia *em trânsito* (TLS) sobre o desempenho de um banco de dados sob uma carga de trabalho real e heterogênea, e discutir o trade-off entre segurança e desempenho. Delimitamos o escopo empírico ao canal de comunicação cliente–servidor de um SGBD orientado a documentos (MongoDB); a criptografia em repouso e em uso é apresentada conceitualmente (Seção 2) e retomada como trabalho futuro (Seção 7), mas não é avaliada empiricamente aqui. Especificamente, buscamos responder:

- **PP1.** Qual é o impacto da criptografia em trânsito (TLS) sobre a latência e a vazão de um SGBD sob uma carga de trabalho real e heterogênea?
- **PP2.** Como esse overhead varia entre diferentes classes de operação (agrupamento, junção, subconsulta, busca, inserção e modificação)?
- **PP3.** Como o overhead *relativo* se relaciona ao custo *intrínseco* de cada consulta?

O restante do artigo está organizado como segue. A Seção 2 apresenta os conceitos fundamentais. A Seção 3 discute o estado da arte. A Seção 4 detalha a metodologia experimental. A Seção 5 apresenta os resultados, discutidos na Seção 6. A Seção 7 conclui o trabalho.

2. Conceitos Fundamentais

Para compreender o impacto de operações criptográficas em bancos de dados distribuídos, é necessário primeiro entender como esses sistemas funcionam, quais mecanismos de criptografia estão disponíveis e como medimos seu desempenho. Esta seção apresenta esses três pilares.

2.1. Bancos de Dados Distribuídos

Um **Banco de Dados Distribuído** (BDD) é uma coleção de bancos de dados logicamente inter-relacionados, fisicamente dispersos por uma rede de computadores e gerenciados por um Sistema de Gerenciamento de Banco de Dados Distribuído (SGBDD) [Elmasri and Navathe 2016]. A motivação para distribuir dados inclui maior disponibilidade, melhor desempenho por proximidade geográfica e escalabilidade horizontal. Contudo, essa distribuição introduz desafios que não existem em sistemas centralizados — em particular, a necessidade de comunicação constante entre nós, que é justamente o canal sobre o qual a criptografia em trânsito atua.

2.1.1. Fragmentação e Replicação

Para distribuir os dados entre os nós, um BDD utiliza duas técnicas complementares: fragmentação e replicação [Elmasri and Navathe 2016].

A **fragmentação** (também chamada de *sharding* ou particionamento) consiste em dividir uma relação em partes menores, cada uma alocada em um nó diferente. Há três abordagens: a *fragmentação horizontal* divide as tuplas segundo algum critério de seleção

(por exemplo, por região geográfica); a *fragmentação vertical* distribui os atributos da relação, como em uma operação de projeção; e a *fragmentação mista* combina ambas as estratégias. A escolha da estratégia influencia diretamente o volume de comunicação entre nós — uma consulta que precisa recombinar fragmentos dispersos gera tráfego de rede, e portanto mais dados a serem cifrados quando TLS está habilitado.

A **replicação**, por sua vez, mantém cópias dos dados em múltiplos nós. Isso melhora a disponibilidade (se um nó falha, outro responde) e o desempenho de leitura (requisições podem ser atendidas localmente). Em contrapartida, escritas precisam ser propagadas para todas as réplicas, gerando tráfego adicional de sincronização. Em um cenário com TLS, cada mensagem de sincronização entre réplicas também passa por cifragem, amplificando o overhead proporcionalmente ao grau de replicação.

2.1.2. Comunicação e Processamento de Consultas

Em um BDD, o processamento de consultas difere fundamentalmente do caso centralizado: o otimizador precisa considerar não apenas o custo de I/O e CPU, mas também o **custo de comunicação** — isto é, o volume de dados transferidos pela rede entre os nós [Elmasri and Navathe 2016]. Estratégias como a semi-junção (*semi-join*) existem justamente para reduzir esse tráfego. Esse custo de comunicação é o ponto central deste trabalho: quando habilitamos TLS, cada byte transmitido entre nós precisa ser cifrado na origem e decifrado no destino, adicionando latência a cada troca de mensagem.

Além disso, para garantir a atomicidade de transações que envolvem múltiplos nós, BDDs utilizam protocolos de coordenação como o *Two-Phase Commit* (2PC), em que um coordenador troca mensagens com todos os participantes para decidir se a transação será confirmada ou abortada [Elmasri and Navathe 2016]. Protocolos de consenso mais modernos, como Raft [Ongaro and Ousterhout 2014], são utilizados em sistemas como Spanner [Corbett et al. 2012]. Todos esses protocolos dependem de múltiplas rodadas de comunicação, o que os torna sensíveis ao overhead de criptografia em trânsito.

2.1.3. Teorema CAP

O teorema CAP [Gilbert and Lynch 2002] formaliza um trade-off fundamental: em um sistema distribuído sujeito a partições de rede, é impossível garantir simultaneamente *Consistência* (todos os nós veem os mesmos dados), *Disponibilidade* (toda requisição recebe resposta) e *Tolerância a Partições*. Como partições são inevitáveis na prática, sistemas precisam optar entre priorizar consistência (modelo CP, como Spanner) ou disponibilidade (modelo AP, como Cassandra). Essa escolha arquitetural determina a quantidade e o padrão de comunicação entre nós, influenciando diretamente o quanto o overhead do TLS se manifesta no desempenho global do sistema.

2.2. Técnicas Criptográficas em Bancos de Dados

A proteção de dados em um banco de dados pode ocorrer em três momentos distintos, de acordo com o estado em que os dados se encontram [Force 2020].

2.2.1. Criptografia em Repouso (*at rest*)

Quando os dados estão armazenados em disco, a criptografia em repouso protege contra acessos não autorizados ao meio físico (roubo de disco, acesso indevido a arquivos do sistema operacional). A abordagem mais comum é a *Transparent Data Encryption* (TDE), que cifra as páginas de dados e os arquivos de log de forma transparente para a aplicação — ou seja, o banco realiza a cifragem e decifragem automaticamente nas operações de I/O, sem alterar as consultas [Force 2020].

O algoritmo predominante é o AES (*Advanced Encryption Standard*), um cifrador de bloco simétrico padronizado pelo NIST [National Institute of Standards and Technology (NIST) 2001], com chaves de 128, 192 ou 256 bits. A granularidade pode variar: criptografia de todo o disco, por *tablespace*, por tabela ou mesmo por coluna individual. O overhead da criptografia em repouso incide sobre cada operação de leitura e escrita em disco, pois os dados precisam ser decifrados ao serem carregados para a memória e cifrados novamente ao serem persistidos.

2.2.2. Criptografia em Trânsito (*in transit*)

A criptografia em trânsito protege os dados enquanto trafegam pela rede — seja entre um cliente e o servidor, seja entre os nós internos do cluster distribuído. O protocolo padrão é o TLS (*Transport Layer Security*) [Rescorla 2018], atualmente na versão 1.3.

O custo de desempenho do TLS pode ser decomposto em dois componentes:

1. **Handshake:** no estabelecimento da conexão, cliente e servidor negociam parâmetros criptográficos utilizando criptografia assimétrica (RSA ou curvas elípticas). É uma operação computacionalmente cara, mas realizada apenas uma vez por conexão (ou por reconexão).
2. **Cifragem simétrica contínua:** após o handshake, todos os dados são cifrados com algoritmos simétricos como AES-GCM ou ChaCha20-Poly1305. O custo por pacote é baixo individualmente, mas em cenários de alta vazão (muitas operações por segundo ou grandes volumes de dados) o impacto acumula.

Em um banco de dados distribuído, o TLS pode ser habilitado tanto na comunicação cliente–servidor quanto na comunicação interna entre os nós do cluster (*intra-cluster*). Neste segundo caso, o impacto tende a ser mais expressivo, pois protocolos de replicação e consenso geram tráfego constante entre os nós.

2.2.3. Criptografia em Uso (*in use*)

Uma linha de pesquisa mais recente busca permitir computações diretamente sobre dados cifrados, eliminando a necessidade de decifrá-los. As abordagens incluem: criptografia determinística e que preserva ordem (OPE) [Boldyreva et al. 2009], que viabilizam consultas de igualdade e intervalos sobre dados cifrados, como explorado pelo sistema CryptDB [Popa et al. 2011]; criptografia totalmente homomórfica (FHE) [Gentry 2009],

que permite computações arbitrárias mas com overhead ainda proibitivo para cargas transacionais; e soluções baseadas em enclaves seguros (*trusted execution environments*), como o Always Encrypted [Antonopoulos et al. 2020]. Embora não sejam o foco dos nossos experimentos, essas técnicas representam o estado da arte em proteção de dados e servem como horizonte para trabalhos futuros.

2.2.4. Integridade

Complementando a confidencialidade, mecanismos de integridade garantem que os dados não foram corrompidos ou adulterados. Técnicas como *hashing* criptográfico (SHA-256), *HMAC* (Hash-based Message Authentication Code) e árvores de Merkle permitem detectar alterações tanto em dados armazenados quanto em dados em trânsito [Force 2020]. No contexto do TLS, a integridade é garantida implicitamente pelos modos de operação AEAD (*Authenticated Encryption with Associated Data*), como o AES-GCM, que combina cifragem e autenticação em uma única operação.

2.3. Métricas de Desempenho

Para avaliar o impacto das operações criptográficas de forma rigorosa, é necessário definir as métricas que serão observadas [Cooper et al. 2010]:

- **Latência:** o tempo entre o envio de uma requisição e o recebimento da resposta correspondente. Como a distribuição de latências tende a ser assimétrica (com uma “cauda longa”), reportamos não apenas a média, mas também a mediana (p50) e os percentis p95 e p99, que revelam o comportamento nos piores casos.
- **Vazão (*throughput*):** o número de operações completadas por unidade de tempo (operações/segundo). Enquanto a latência captura a experiência individual de cada requisição, a vazão reflete a capacidade agregada do sistema sob carga.
- **Utilização de CPU:** a fração do processamento dedicada a operações criptográficas. Permite distinguir se o overhead observado decorre de cifragem ou de outros fatores (I/O, contenção de rede).
- **Overhead de rede:** o aumento no volume de bytes transmitidos devido aos cabeçalhos e metadados introduzidos pelo TLS (registros TLS, MACs, padding).

Para quantificar o impacto relativo da criptografia, definimos o **overhead** em relação ao cenário *baseline* (sem criptografia):

$$\text{overhead}(\%) = \frac{M_{\text{cripto}} - M_{\text{baseline}}}{M_{\text{baseline}}} \times 100 \quad (1)$$

onde M representa o valor de uma métrica (latência ou vazão) no respectivo cenário. Um overhead positivo em latência indica degradação; um overhead negativo em vazão indica queda na capacidade do sistema. Neste trabalho, as métricas efetivamente coletadas são a **latência** (com ênfase na mediana, p50) e a **vazão**; a utilização de CPU e o overhead de rede são definidos acima por completude conceitual, mas não foram instrumentados em nosso arcabouço de medição (Seção 4).

3. Trabalhos Relacionados

Esta seção situa nosso estudo em relação a (i) trabalhos teóricos e de sistemas sobre proteção criptográfica de bancos de dados e (ii) ferramentas de software livre relevantes para o experimento.

3.1. Estudos Teóricos e de Sistemas

Boa parte da literatura sobre criptografia em bancos de dados concentra-se na proteção dos dados *em repouso* e *em uso*, buscando permitir consultas diretamente sobre dados cifrados. O CryptDB [Popa et al. 2011] foi um marco nessa direção: combinando criptografia determinística e que preserva ordem (OPE) [Boldyreva et al. 2009], executa consultas SQL de igualdade e intervalo sobre dados cifrados, reportando um overhead de vazão da ordem de 20–30% em cargas transacionais. O *Always Encrypted* [Antonopoulos et al. 2020] adota abordagem complementar, apoiando-se em enclaves seguros (*trusted execution environments*) para processar dados decifrados apenas dentro de uma região protegida do processador. No extremo da expressividade está a criptografia totalmente homomórfica (FHE) [Gentry 2009], que admite computações arbitrárias sobre cifras, mas cujo custo ainda é proibitivo para cargas de produção. A systematization of knowledge de Fuller et al. [Fuller et al. 2017] organiza esse espaço de projeto, evidenciando o trade-off recorrente entre expressividade das consultas, vazamento de informação e desempenho.

Em contraste, a criptografia *em trânsito* (TLS) [Rescorla 2018] costuma ser tratada como um custo secundário e “barato”, raramente quantificado de forma sistemática na literatura de *benchmarking* de SGBDs. Nosso trabalho examina justamente essa suposição, medindo empiricamente o custo do TLS no canal cliente–servidor de um SGBD orientado a documentos.

3.2. Ferramentas de Software Livre

Conforme sugerido no enunciado, priorizamos ferramentas de software livre, que podem ser instaladas e testadas sem restrições. No que diz respeito a SGBDs, o **MongoDB Community** oferece suporte nativo a TLS tanto no canal cliente–servidor quanto na comunicação intra-cluster [MongoDB, Inc. 20XX], o que o torna adequado ao nosso experimento. Entre os SGBDs relacionais distribuídos (*NewSQL*), CockroachDB e YugabyteDB também oferecem TLS intra-cluster e criptografia em repouso, enquanto o Apache Cassandra segue o modelo AP do teorema CAP [Gilbert and Lynch 2002]. A cifragem em si, em todos esses sistemas, é tipicamente delegada a bibliotecas como o OpenSSL e acelerada por instruções de hardware (AES-NI).

Para a geração de carga, ferramentas consolidadas incluem o YCSB [Cooper et al. 2010], voltado a sistemas NoSQL, além de sysbench e BenchBase (TPC-C). Optamos, contudo, por uma carga composta de *consultas reais* herdadas de um projeto anterior (Seção 4.2), em vez de um gerador sintético: essa escolha sacrifica a padronização típica dos geradores em favor de maior validade ecológica, capturando padrões de acesso heterogêneos (analíticos e transacionais).

3.3. Posicionamento

Os *benchmarks* clássicos tendem a enfatizar a criptografia em repouso ou consultável sobre cargas sintéticas e uniformes, nas quais todas as operações têm custo semelhante. Nosso estudo difere em dois pontos. Primeiro, isola o canal *em trânsito* (TLS) de um repositório de documentos sob uma mistura heterogênea de consultas reais — analíticas (agrupamento, junção, subconsulta) e transacionais (busca, inserção, modificação).

Segundo, caracteriza como o overhead *relativo* depende do custo *intrínseco* de cada consulta — dimensão que cargas sintéticas uniformes tendem a ocultar, mas que se mostra central para interpretar corretamente o impacto da criptografia.

4. Metodologia

O método empregado neste estudo é o **experimento** controlado. Descrevemos a seguir, com o detalhe necessário à reprodução, o ambiente, a carga de trabalho, os cenários e o protocolo de medição. Todo o código e os dados citados encontram-se no repositório do projeto.

4.1. Ambiente Experimental

Os experimentos foram executados em um único *host* físico — um computador (processador Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz (1.99 GHz), 16 GB de RAM) com sistema operacional Windows 11 com WSL2. A orquestração da infraestrutura é feita via **Docker Compose**, que instancia duas topologias distribuídas de alta disponibilidade do SGBD **MongoDB**, a partir da imagem oficial `mongo` (versão `latest`):

- **Cenário Baseline (`rs_baseline`):** um *Replica Set* composto por três contêineres interdependentes (`mongo_base_1`, `mongo_base_2`, `mongo_base_3`) mapeados, respectivamente, nas portas 27017, 27018 e 27019 do hospedeiro, operando sem criptografia.
- **Cenário TLS (`rs_tls`):** um segundo *Replica Set* com três contêineres (`mongo_tls_1`, `mongo_tls_2`, `mongo_tls_3`) expostos nas portas 27020, 27021 e 27022. Estes foram inicializados com a diretiva `--tlsMode requireTLS` e um par certificado/chave *self-signed*, exigindo cifragem de canal, mas dispensando autenticação mútua por certificado de cliente (`--tlsAllowConnectionsWithoutCertificates`).

O cliente utilizado é o `mongosh` (*MongoDB Shell*), acionado sob demanda pelo script de automação Python. Como a topologia foi isolada em uma rede virtual dedicada (`mongo_net`), a injeção da carga de trabalho ocorre interativamente via `docker exec` diretamente na entrada padrão (STDIN) do contêiner primário. Dessa forma, elimina-se o ruído do hospedeiro e da resolução de DNS, garantindo que o cronômetro meça predominantemente o **custo computacional** da criptografia (*handshake* e cifra) somado ao tempo de descoberta da topologia do *cluster*, e não flutuações de uma rede de longa distância (WAN). Um contêiner adicional (PostgreSQL 15) foi usado estritamente para extrair e converter o conjunto de dados na etapa inicial (Seção 4.2), não participando das medições de desempenho.

É fundamental delimitar a evolução do escopo: a adoção de *Replica Sets* garante que a carga sofra a amplificação natural do ambiente distribuído (mecanismos de consenso e replicação de escrita *intra-cluster*). Contudo, como todos os contêineres residem no mesmo equipamento físico via ponte de rede local, medimos o canal seguro de forma isolada, sem o acréscimo da latência de comutadores físicos externos. Essa característica do arcabouço é analisada como uma ameaça à validade externa na Seção 6.

4.2. Carga de Trabalho

Conjunto de dados. Utilizamos o conjunto de dados público do *Stack Exchange Data Dump* [Stack Exchange, Inc. 20XX], herdado de um projeto anterior da disciplina. Originalmente relacional, o dump foi extraído de um PostgreSQL e convertido em coleções MongoDB. A redução foi uma decisão metodológica para garantir que o *working set* coubesse integralmente na memória RAM da máquina de testes. Se a base excedesse a RAM, o sistema operacional faria paginação em disco e a latência brutal de leitura do disco ofuscaria completamente o tempo de processamento criptográfico do TLS. Reduzir a base garantiu que o gargalo fosse estritamente de CPU/Criptografia, não de I/O de disco. A Tabela 1 resume o volume de dados. Os mesmos índices secundários foram criados em ambas as instâncias (por exemplo, sobre `Id`, `OwnerUserId`, `PostId`, `UserId`, `Name`, `ViewCount` e `Score`).

Tabela 1. Volume do conjunto de dados (coleções MongoDB), idêntico nos dois cenários.

Coleção	Documentos
Posts	100 000
Comments	33 126
Badges	224 671
Votes	89 353
Users	15 017
PostLinks	435
PostTypes	8

Consultas. Em vez de um gerador sintético, a carga é composta de **30 consultas reais** (também herdadas do projeto anterior), organizadas em seis classes que cobrem tanto operações analíticas quanto transacionais. A Tabela 2 descreve as classes e sua quantidade. As mesmas 30 consultas, na mesma ordem, são executadas em ambos os cenários.

Tabela 2. Classes de consulta que compõem a carga de trabalho (30 consultas ao todo).

Classe	Descrição / exemplo	Qtd.
Agrupamento	Agregações analíticas (<code>\$group</code> , <code>\$avg</code> , <code>\$sum</code>)	6
Junção	Junções entre coleções (<code>\$lookup</code> , até 4 níveis)	7
Subconsulta	Filtros correlacionados (<code>\$expr</code> , <i>pipelines</i> internos)	5
Busca	Busca por chave e por padrão textual (<code>find</code> , <code>\$regex</code>)	2
Inserção	Inserções individuais e em lote (<code>insertOne</code> , <code>insertMany</code>)	5
Modificação	Atualizações e remoções (<code>updateMany</code> , <code>deleteMany</code>)	5
Total		30

O cliente é *único* e as consultas são executadas de forma *sequencial* (uma repetição por vez), de modo que a vazão medida reflete a capacidade sob um cliente serial, e não sob concorrência (ver Seção 4.5).

4.3. Estratégia de Indexação

Para o experimento, aplicou-se índices B-Tree em todas as chaves primárias (`Id`) e estrangeiras (`OwnerId`, `PostId`), além de índices compostos em campos de filtragem e ordenação (como `PostTypeId` combinado com `Score` ou `ViewCount`).

A criação de índices foi fundamental para eliminar *Collection Scans* e garante que o motor do banco de dados resolva as junções e agregações com complexidade de tempo otimizada. Ao remover a ineficiência algorítmica da consulta, o *overhead* medido representa puramente o custo computacional do protocolo TLS.

4.4. Cenários Avaliados

Avaliamos dois cenários que diferem *exclusivamente* quanto à ativação da criptografia em trânsito no canal de comunicação (Tabela 3). Para isolar o custo computacional do TLS, ambos os cenários foram implantados utilizando uma arquitetura distribuída em *Replica Set* contendo três nós interdependentes (um nó Primário e dois Secundários), isolados dentro de uma rede virtual dedicada do Docker.

Manter a topologia de cluster idêntica, bem como o *host* hospedeiro, o *dataset* reduzido, a estratégia de indexação B-Tree e a ordem de execução das consultas, é o que garante o rigoroso controle de variáveis do experimento. Dessa forma, neutralizou-se o impacto do mecanismo de consenso e replicação de dados interno do banco, isolando com precisão o overhead simétrico e assimétrico imposto pelo protocolo criptográfico. A criptografia em repouso (TDE) e em uso, embora conceitualmente relevantes (Seção 2), **não são avaliadas empiricamente** neste estudo e ficam como trabalho futuro (Seção 7).

Tabela 3. Cenários avaliados sob arquitetura distribuída de *Replica Set*.

Cenário	Em trânsito (TLS)	Topologia Distribuída e Configuração
C0 (Baseline)	Não	<i>Replica Set</i> (<code>rs_baseline</code>) composto por 3 nós contêineres (<code>mongo_base_1</code> , <code>mongo_base_2</code> , <code>mongo_base_3</code>) operando internamente na porta padrão 27017.
C1 (TLS)	Sim	<i>Replica Set</i> (<code>rs_tls</code>) composto por 3 nós contêineres (<code>mongo_tls_1</code> , <code>mongo_tls_2</code> , <code>mongo_tls_3</code>) operando na porta 27017 com diretivas de <code>requireTLS</code> e cifragem ativa.

4.5. Protocolo de Medição

A medição é orquestrada por um script de automação. Para *cada uma* das 30 consultas e *cada* cenário, o procedimento é:

1. **Aquecimento (*warm-up*):** uma execução descartada, que aquece o *cache* do sistema operacional e o motor de armazenamento WiredTiger;
2. **Medição:** 20 execuções cronometradas.

Cada execução consiste em acionar o `mongosh` interativamente de dentro do container da aplicação via `docker exec`, injetando a consulta diretamente pela entrada padrão (STDIN). Esse procedimento abre uma *nova conexão* ao `mongod`, executa a consulta na arquitetura de *Replica Set* e encerra. Cronometra-se o tempo de parede (*wall-clock*) da invocação completa. A partir das 20 amostras, calculam-se, por consulta e cenário: a latência mediana (p50), os percentis de cauda (p95, p99) e a vazão, definida como $n / \sum_i t_i$ — isto é, o *recíproco da latência média* sob um único cliente sequencial. O overhead relativo é obtido pela Equação (1), sempre em relação ao baseline (C0).

Três características do arcabouço merecem registro explícito, por afetarem a interpretação dos resultados:

- **Conexão nova por execução.** Como cada repetição abre uma conexão nova, no cenário C1 o *handshake* TLS é pago a *cada* execução. Isso corresponde ao *pior caso* para o custo do handshake — ou seja, a ausência de reúso de conexões (*connection pooling*).
- **Partida do mongosh.** O tempo absoluto inclui a inicialização do processo `mongosh` (ambiente Node.js), o que infla os valores absolutos de latência (é por isso que mesmo a consulta mais barata leva $\approx 0,6$ s, tempo que engloba o acionamento do serviço Docker e a inicialização do motor Node.js interno). Contudo, essa parcela é *comum* aos dois cenários e, portanto, *cancela-se na diferença* baseline–TLS. Assim, embora os valores absolutos não representem a latência pura da consulta, a *diferença* entre os cenários isola legitimamente o custo do TLS (handshake e cifra).
- **Estimadores de cauda com $n = 20$.** Os índices usados para p95 e p99 ($\lfloor 20 \cdot 0,95 \rfloor = \lfloor 20 \cdot 0,99 \rfloor = 19$) coincidem, de modo que p95 e p99 correspondem *ambos ao máximo* observado — razão pela qual são idênticos em nossos dados. Por isso, adotamos a **mediana (p50)** como métrica principal de latência e tratamos p95/p99 apenas como um indicador de cauda (o valor máximo), com essa ressalva.

A reprodutibilidade é favorecida pelo caráter determinístico do experimento: o mesmo dataset é carregado em ambas as instâncias, as 30 consultas são fixas e executadas na mesma ordem nos dois cenários, e o código de automação está disponível no repositório.

5. Experimentos e Resultados

Habilitar o TLS degradou o desempenho em *todas* as 30 consultas, sem exceção. Considerando o conjunto completo sob a arquitetura otimizada e indexada, o overhead de latência (p50) foi de **16,5% em média** (mediana de 16,7%), variando de 11,8% a 18,4% conforme a consulta; a vazão caiu, correspondentemente, **14,2% em média** (mediana de 14,5%). A Tabela 4 agrega esses resultados por classe de consulta.

Latência e vazão por classe A Figura 1 contrasta a latência p50 média de cada classe nos dois cenários, e a Figura 2 faz o mesmo para a vazão. Devido à indexação do banco de dados, o tempo intrínseco de processamento de dados foi mitigado. Em ambas, a barra do cenário TLS é consistentemente pior, mas a distância entre as barras é pequena e razoavelmente uniforme — um primeiro indício de que o custo do TLS é aproximadamente *aditivo*, e não proporcional ao trabalho da consulta.

Tabela 4. Overhead do TLS por classe de consulta. Overhead de latência calculado sobre a p50; queda de vazão em relação ao baseline.

Classe	Consultas	Overhead p50 (%)	Queda de vazão (%)
Agrupamento	6	16,7	15,0
Busca	2	12,7	8,9
Inserção	5	17,3	16,7
Junção	7	16,4	15,8
Modificação	5	17,2	13,1
Subconsulta	5	16,4	12,1
Geral	30	16,5	14,2

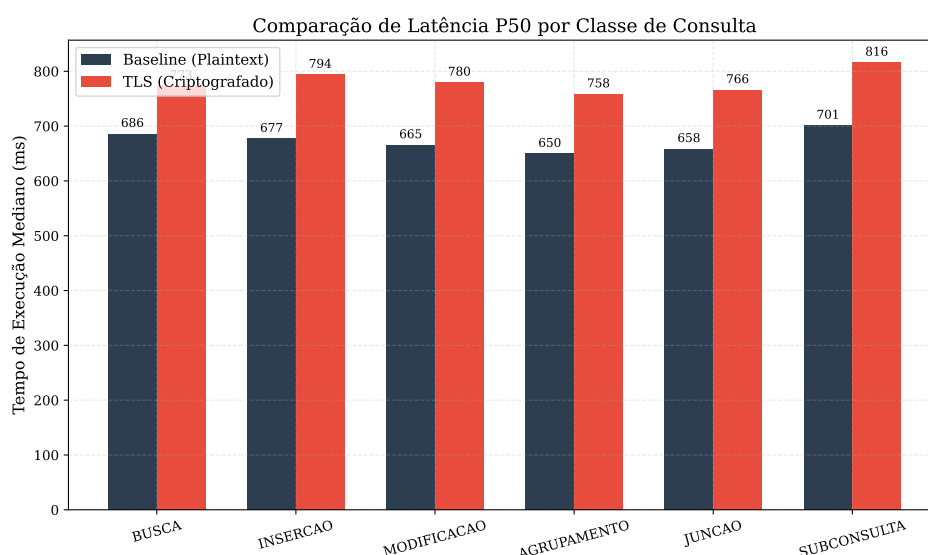


Figura 1. Latência p50 média por classe de consulta, baseline versus TLS.

O custo do TLS é quase constante A observação central do estudo aparece ao examinar o overhead em termos *absolutos*: a diferença de latência p50 entre TLS e baseline é notavelmente estável, com **mediana de 110,6 ms** por consulta (mínimo de 79,6 ms). Esse valor quase fixo é a assinatura do *handshake* TLS pago a cada conexão (Seção 4.5): como cada execução abre uma conexão nova, o custo de estabelecimento do canal seguro incide uma vez por consulta, praticamente independente do trabalho que a consulta realiza.

A consequência direta é que o overhead *relativo* apresenta uma variação extremamente controlada — ele diminui sutilmente conforme cresce o custo intrínseco da consulta, pois o custo fixo do *handshake* é amortizado por um tempo de execução marginalmente maior. Com a indexação exaustiva eliminando os gargalos algorítmicos, os tempos de baseline foram compactados na faixa de 0,63 s a 0,73 s, fazendo com que o overhead relativo oscilasse estritamente entre **11,8% e 18,4%**.

Sob essa nova dinâmica, as consultas estruturalmente mais simples (como buscas e inserções) sofrem um impacto percentual ligeiramente maior (média de 17,3% em inserções), ao passo que as consultas analiticamente mais densas (como as junções complexas

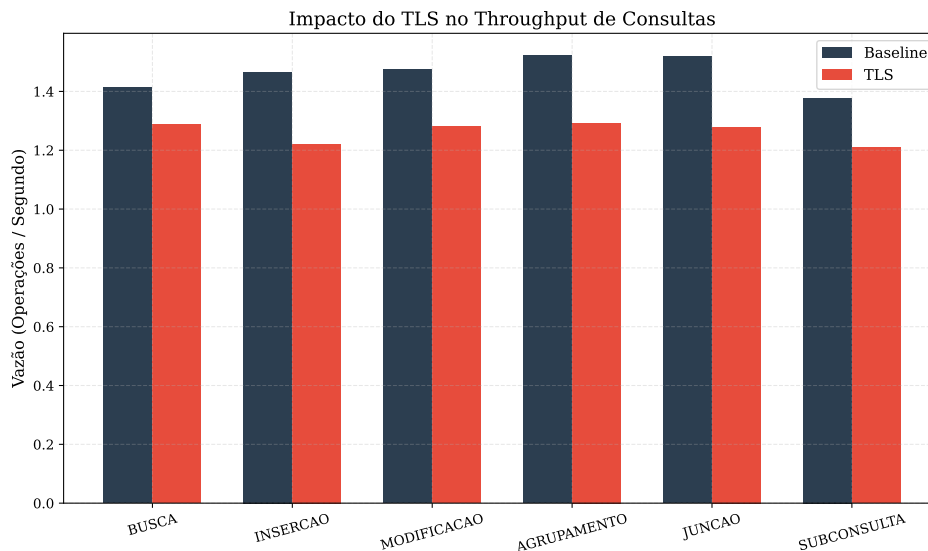


Figura 2. Vazão média (operações por segundo) por classe de consulta, baseline versus TLS.

e subconsultas) conseguem amortizar melhor esse impacto absoluto. Em outras palavras, o *mesmo* custo absoluto de segurança (com mediana de 110,6 ms) passa a representar a quase totalidade do overhead perceptível pelo cliente quando a eficiência do banco de dados é maximizada.

A consequência direta é que o overhead *relativo* não é uniforme — ele *diminui* conforme cresce o custo intrínseco da consulta, pois o custo fixo do handshake é amortizado por um tempo de execução maior. A Figura 3 torna esse efeito explícito: consultas baratas (busca, inserção, junções e subconsultas simples), com p50 baseline em torno de 0,63–0,64 s, sofrem overhead de 16–17%, ao passo que as duas consultas mais custosas — uma junção de múltiplos `$lookup` (p50 baseline de 0,73 s) e uma subconsulta correlacionada (0,69 s) — caem para 14,7% e 15,8%, respectivamente. Em outras palavras, o *mesmo* custo absoluto de segurança representa uma fração cada vez menor do tempo total à medida que a consulta se torna mais pesada.

Resultados detalhados A Tabela 5 apresenta, para as 30 consultas, a latência p50 nos dois cenários, o overhead relativo e a queda de vazão.

6. Discussão

6.1. O trade-off segurança versus desempenho

Os resultados sugerem que, mesmo em uma arquitetura distribuída rigorosamente indexada, o custo do TLS é *moderado e previsível*. Um overhead mediano em torno de 16,5% em latência e de 14,2% em vazão é, para a maioria das aplicações, um preço aceitável pela confidencialidade e integridade que o TLS proporciona — especialmente diante de exigências regulatórias como a LGPD e o GDPR. Mais importante do que o número absoluto é o seu *padrão*: por ser aproximadamente aditivo, o custo do TLS é fácil de prever e de planejar em termos de capacidade.

Tabela 5. Resultados por consulta: latência p50 (s) no baseline e com TLS, overhead de latência (%) e queda de vazão (%).

Classe	#	p50 base (s)	p50 TLS (s)	Overhead (%)	Queda vazão (%)
Agrupamento	1	0,648	0,754	16,4	18,0
Agrupamento	2	0,641	0,752	17,3	14,6
Agrupamento	3	0,636	0,745	17,2	14,5
Agrupamento	4	0,645	0,752	16,6	18,2
Agrupamento	5	0,651	0,763	17,1	10,7
Agrupamento	6	0,677	0,783	15,8	14,1
Junção	1	0,641	0,743	15,9	14,8
Junção	2	0,642	0,747	16,4	15,0
Junção	3	0,644	0,750	16,5	14,3
Junção	4	0,635	0,740	16,4	18,6
Junção	5	0,638	0,747	17,1	18,4
Junção	6	0,679	0,797	17,4	15,8
Junção	7	0,729	0,836	14,7	13,5
Subconsulta	1	0,703	0,811	15,3	12,9
Subconsulta	2	0,728	0,852	16,9	8,8
Subconsulta	3	0,692	0,802	15,8	14,4
Subconsulta	4	0,681	0,800	17,6	10,0
Subconsulta	5	0,701	0,816	16,4	14,1
Busca	1	0,673	0,753	11,8	5,3
Busca	2	0,698	0,794	13,6	12,6
Inserção	1	0,681	0,802	17,7	18,0
Inserção	2	0,674	0,790	17,2	15,3
Inserção	3	0,669	0,789	18,0	18,2
Inserção	4	0,683	0,797	16,8	17,9
Inserção	5	0,677	0,791	16,8	14,1
Modificação	1	0,656	0,777	18,4	15,7
Modificação	2	0,665	0,781	17,4	14,8
Modificação	3	0,666	0,777	16,6	10,2
Modificação	4	0,664	0,775	16,7	14,3
Modificação	5	0,675	0,788	16,8	10,2

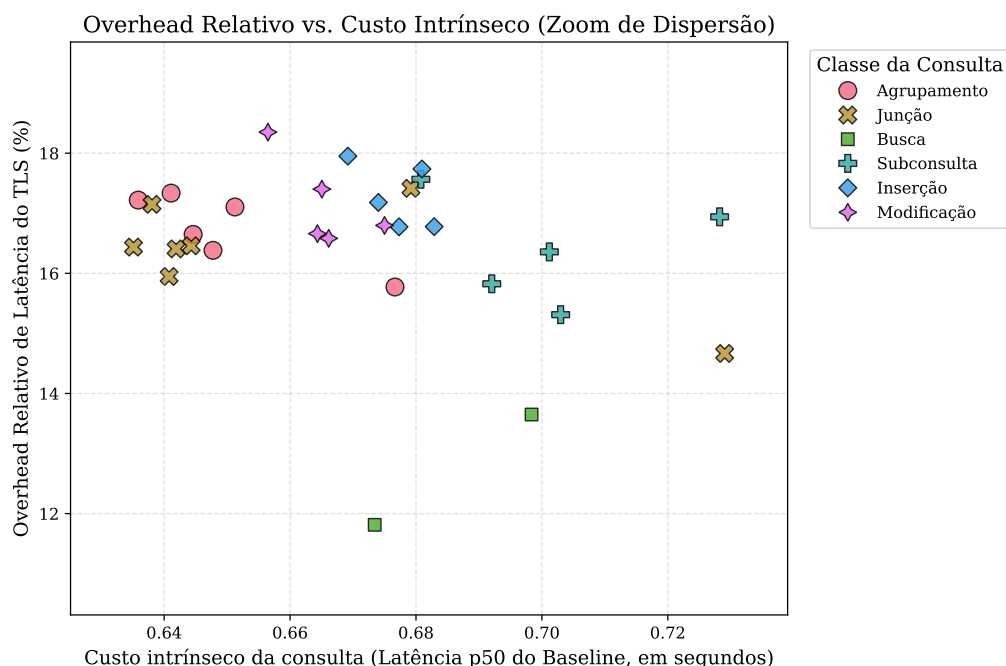


Figura 3. Overhead relativo de latência do TLS (%) em função do custo intrínseco da consulta (latência p50 do baseline). Cada ponto é uma das 30 consultas.

6.2. Qual componente domina o overhead?

A evidência aponta de forma consistente para o **handshake por conexão** como componente dominante, e não para a cifragem simétrica contínua dos dados. Dois fatos sustentam essa conclusão: (i) o custo absoluto adicional é quase constante (mediana de 110,6 ms), independentemente do volume de dados que a consulta processa ou retorna; e (ii) o overhead relativo decresce com o custo da consulta (Figura 3). Se a cifra simétrica sobre os dados transferidos fosse o fator dominante, esperaríamos o oposto — consultas que retornam mais dados pagariam *mais*, não menos. A cifra em si, acelerada por instruções de hardware (AES-NI) e barata por *byte*, é desprezível diante do custo assimétrico do estabelecimento da conexão para os tamanhos de resultado desta carga.

6.3. Implicações práticas

A principal implicação é que **o número que medimos representa um pior caso**. Como nosso arcabouço abre uma conexão nova a cada execução, ele paga o handshake integralmente em toda operação. Aplicações reais mantêm *pools* de conexões persistentes e reutilizam o canal já estabelecido por milhares de operações, amortizando o handshake até torná-lo negligenciável. Assim, sistemas com reúso de conexão devem observar overhead bem menor do que o aqui reportado, enquanto os padrões que mais sofrem são justamente os de *conexões curtas* — funções *serverless*, scripts de linha de comando (como o próprio *mongosh*) e arquiteturas de conexão-por-requisição. Mitigações adicionais incluem o TLS 1.3, cujo handshake de 1-RTT e a retomada de sessão (*session resumption*) reduzem ainda mais esse custo [Rescorla 2018].

6.4. Ameaças à validade

Reconhecemos limitações importantes, coerentes com o escopo delimitado do estudo.

Validade interna. A latência absoluta inclui a partida do processo `mongosh`; embora essa parcela se cancele na diferença entre cenários (Seção 4.5), ela impede a leitura dos valores absolutos como latência “pura” de consulta. O número de repetições ($n = 20$) é modesto e faz os estimadores de p95/p99 colapsarem no máximo observado, razão pela qual nos apoiamos na mediana. Por fim, caso a imagem do MongoDB não tenha sido fixada em uma versão específica (isto é, usando a tag `latest`), a reprodutibilidade bit a bit fica comprometida.

Validade externa. O experimento mitigou a limitação de instâncias isoladas (*standalone*) ao adotar uma arquitetura distribuída real de *Replica Set* contendo três nós ativos. Contudo, esses contêineres operam isolados dentro de uma rede virtual hospedada sobre um mesmo *host* físico. Isso significa que a ausência de latência de rede real (como o roteamento físico de pacotes em redes LAN/WAN) faz com que a fração de tempo atribuível à criptografia (custo de CPU) ganhe um peso proporcionalmente maior na métrica do que teria em uma nuvem geograficamente distribuída. Além disso, avaliamos apenas a criptografia *em trânsito*: os custos de criptografia *em repouso* (TDE) e *em uso* não foram medidos. O certificado é *self-signed* e a autenticação mútua está desabilitada, de modo que não contabilizamos o custo de validação de cadeias de certificados nem de mTLS, presentes em ambientes de produção. Finalmente, os resultados referem-se a um único conjunto de dados (Stack Exchange) e a um único *host*; cargas, esquemas e hardware distintos podem deslocar a magnitude dos números absolutos, mantendo a tendência demonstrada.

7. Conclusão e Trabalhos Futuros

Este trabalho quantificou, de forma reprodutível, o overhead da criptografia em trânsito (TLS) sobre o desempenho do MongoDB, sob uma carga de 30 consultas reais e heterogêneas. Retomando as perguntas de pesquisa:

- **PP1.** Habilitar o TLS na comunicação entre o cliente e o cluster distribuído (*Replica Set*) degradou o desempenho em todas as consultas, com overhead mediano de latência de cerca de 16,7% (16,5% em média) e queda de vazão de cerca de 14,5% (14,2% em média) — um custo moderado e previsível para a introdução de segurança em nós distribuídos.
- **PP2.** O overhead variou entre as classes (de $\sim 12,7\%$ em busca a $\sim 17,3\%$ em inserção), mas essa variação *notadamente* não decorre da topologia distribuída ou da semântica da classe: decorre estritamente do custo típico de processamento das consultas de cada classe sobre os dados indexados.
- **PP3.** O overhead *relativo* é inversamente proporcional ao custo intrínseco da consulta. Isso porque o custo do TLS para a inicialização de sessão no ecossistema distribuído é quase constante em termos absolutos ($\approx 110,6$ ms), composto pelo *handshake* criptográfico combinado à descoberta de topologia do *Replica Set* pagos a cada nova conexão; quanto mais otimizada a consulta no nó primário, mais esse custo fixo de rede se destaca.

A mensagem prática é dupla: o TLS em trânsito é barato o bastante para ser a opção padrão na maioria dos cenários; e o reúso de conexões (*connection pooling*) é a alavanca mais eficaz para reduzir seu custo, já que ataca diretamente o componente dominante.

Trabalhos futuros. As limitações apontadas na Seção 6 delineiam extensões naturais. Primeiro, avaliar a criptografia *em repouso* (por exemplo, a cifragem no nível do Wired-Tiger ou via LUKS no volume de dados), completando os cenários originalmente previstos. Segundo, um estudo específico do efeito do *connection pooling*, contrastando o pior caso aqui medido com o custo amortizado. Por fim, avançar para a criptografia *em uso*, avaliando recursos como a *Queryable Encryption* do próprio MongoDB, e replicar o experimento em outros SGBDs distribuídos e sobre uma rede real.

Referências

- Antonopoulos, P., Arasu, A., Singh, K. D., Eguro, K., Gupta, N., Jain, R., Kaushik, R., Kodavalla, H., Kossmann, D., Ramamurthy, R., Szymaszek, J., Trimmer, J., Vaswani, K., Venkatesan, R., and Zwillig, M. (2020). Azure SQL database always encrypted. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1511–1525. ACM.
- Boldyreva, A., Chenette, N., Lee, Y., and O’Neill, A. (2009). Order-preserving symmetric encryption. In *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Computer Science*, pages 224–241. Springer.
- Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., and Sears, R. (2010). Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154. ACM.
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., et al. (2012). Spanner: Google’s globally-distributed database. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 261–264. USENIX Association.
- Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, 7 edition.
- Force, J. T. (2020). Security and privacy controls for information systems and organizations. Special Publication SP 800-53 Rev. 5, National Institute of Standards and Technology (NIST). <https://doi.org/10.6028/NIST.SP.800-53r5>.
- Fuller, B., Varia, M., Yerukhimovich, A., Shen, E., Hamlin, A., Gadepally, V., Shay, R., Mitchell, J. D., and Cunningham, R. K. (2017). SoK: Cryptographically protected database search. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 172–191. IEEE.
- Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178. ACM.
- Gilbert, S. and Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59.
- MongoDB, Inc. (20XX). Configure mongod and mongos for TLS/SSL. MongoDB Manual. <https://www.mongodb.com/docs/manual/tutorial/configure-ssl/>.

- National Institute of Standards and Technology (NIST) (2001). Advanced encryption standard (AES). Federal Information Processing Standards Publication FIPS PUB 197, NIST.
- Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, pages 305–319. USENIX Association.
- Popa, R. A., Redfield, C. M. S., Zeldovich, N., and Balakrishnan, H. (2011). CryptDB: Protecting confidentiality with encrypted query processing. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, pages 85–100. ACM.
- Rescorla, E. (2018). The transport layer security (TLS) protocol version 1.3. RFC 8446, Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc8446>.
- Stack Exchange, Inc. (20XX). Stack exchange data dump. Internet Archive. <https://archive.org/details/stackexchange>.